

Snake Game Project: Chat History

You (message 1)

[Uploaded file: 45_snake-main.zip]

This is a pygame Snake game (files attached). If I tap the opposite arrow key while moving, e.g. Left while moving Right, the snake dies instantly. Quick double turns like Up then Left also kill it. Find the root cause and fix it so direction changes and wall and self collisions are fair. Give me the full updated files.

Claude

I fixed the instant deaths by changing only `game/snake.py`, `main.py`, `game_engine.py`, `food.py` and the README are unchanged, so they stay as you have them. Headless tests of the snake logic passed, but I haven't played the game with a real keyboard.

Root cause

1. `set_direction()` **had no reverse check**. Pressing Left while moving Right set the direction to `(-1, 0)`. The next move put the head on the neck segment, and `collides_with_self()` correctly reported a hit.
2. **Direction was changed on every key press, not once per move**. The game moves about 8 times a second but checks keys every frame. So a quick Up then Left before the next move turned the snake straight back into itself, even though each turn looked legal against the other.

The fix

- Snake now remembers the direction of its last real move (`last_move_direction`).

- Turns go into a small queue of at most 2, and `move()` applies one per step. Quick double turns like Up then Left now play out as two separate moves.
- `set_direction()` checks each new turn against the last queued turn, or the last real move if the queue is empty. Reversals and repeats are ignored, so tapping the opposite key does nothing.

I did not change the collision checks, because they were already correct. The tail is removed before the self-check, so moving into the cell the tail is leaving is fair. A snake that is still growing correctly collides with its tail. Wall collision is unchanged.

Tests

- Tapping the opposite key is ignored and the snake survives.
 - Up then Left in the same tick works.
 - Up, Left, Down in quick succession works.
 - A real self-collision still kills the snake.
 - Chasing your own tail without growing is safe.
 - The wall still kills the snake.
 - The `GameEngine` runs normally with the updated snake.
-

You (message 2)

Replace the console print on game over with an on-screen Game Over screen showing the final score. It should wait for input rather than closing or freezing. Keep the existing code structure.

Claude

The game now shows an on-screen Game Over screen with the final score, and the console print is gone. The only file that changed this time is `game/game_engine.py`. I ran a headless test and looked at a rendered frame, but I haven't played it with a real keyboard.

When the snake hits a wall or itself, the final board freezes and dims. "GAME OVER", "Final Score: N" and "Press any key to exit" appear on top. The window stays open and responsive until you press a key, then it closes through the normal quit path.

I also added a 0.7-second delay before key presses count on that screen. Without it, a direction key you were still pressing when you died would close the window immediately. The constant

`GAME_OVER_INPUT_DELAY_MS` controls this.

- `_end_game()`: a small helper that sets `game_over` and records the time, used in both collision branches.
- `handle_keydown()`: on the Game Over screen it now ignores direction keys and quits on any key after the delay. Otherwise it is unchanged.
- `render()`: the print is replaced by a call to the new `_render_game_over()`.
- `_game_over_logged`: removed, since nothing uses it any more.

"Press any key to exit" is a placeholder. Task 3 will replace it with the replay and difficulty choice.

`main.py`, `snake.py` and `food.py` are unchanged.

You (message 3)

On the Game Over screen, let me choose Easy, Medium or Hard speed (keys 1, 2, 3) to play again, or press Esc/Q to exit the game. Make sure arrow keys pressed at the moment of death can't accidentally skip the screen.

Claude

The Game Over screen now lets you restart at a chosen speed with 1, 2 or 3, or exit with Esc or Q. Only `game/game_engine.py` changed. I tested this headlessly and checked a rendered frame, but I haven't played it with a real keyboard.

Speeds

- **1 Easy:** 6 moves per second.
- **2 Medium:** 10 moves per second.
- **3 Hard:** 15 moves per second.

These are in the `DIFFICULTIES` dict at the top of the file, so you can retune them. I picked values that divide 60 evenly, because the engine turns speed into whole frames per move.

Arrow keys can't skip the screen. Two safeguards:

- On Game Over, the only keys that do anything are 1, 2, 3, Esc and Q. Arrows, WASD and everything else are ignored.
- For 0.7 seconds after death, all keys are ignored, including those five, so a stray press can't pick a difficulty or exit by accident.

What changed

- `restart(moves_per_second)`: this new method builds a fresh snake, sets the score to 0, sets the chosen speed and clears game over. The food is respawned clear of the new snake, so it can't land on it.

- `handle_keydown()`: the Game Over branch now does the key filtering above. Normal gameplay controls are untouched.
- `_render_game_over()`: it shows the difficulty choices and the exit hint in place of the old "Press any key to exit" line.

The first round still starts at the original speed of 8 moves per second. The 1/2/3 choice only applies from the first restart onward.

`main.py`, `snake.py` and `food.py` are unchanged. The earlier `snake.py` fix is still the one to keep.

You (message 4)

Add sound effects for eating food and for game over. I don't have audio files, so generate the sounds in code, and make it still run if no audio device is available.

Claude

Eating food and game over now have sound effects, and the game still runs if there's no audio device. I tested both cases headlessly, but I haven't heard the sounds on a real speaker, so you'll want to check how they sound.

You need one new file and one updated file:

- `game/sound.py` is new. It holds a `SoundEffects` class that builds both sounds from math at startup, so there are no audio files. It needs no extra packages beyond `pygame`.
- `game/game_engine.py` is updated. It imports `SoundEffects` and creates one in `__init__`. It plays the eat sound when food is eaten and the game-over sound once in `_end_game()`. Nothing else changed.

The eat sound is a short, bright two-note blip that rises in pitch (about 0.15 s). The game-over sound is a slower four-note descending tone (about 1 s). Both play every time, including after a restart from the Game Over screen.

No audio device. If the mixer can't start, `SoundEffects` turns itself off and its play methods do nothing. I tested this by forcing a nonexistent audio driver. The mixer failed to start, and the game still ran, scored, died, and restarted normally. Any error while playing a sound is also swallowed, so a sound problem can't crash the game. The catch is broad on purpose, so a real bug in `sound.py` would fail silently rather than show an error.

My earlier tests still pass, and `main.py` and `food.py` are byte-for-byte identical to your originals.

Your folder should now contain `main.py`, `game/snake.py` (the Task 1 fix), `game/game_engine.py`, `game/food.py`, and the new `game/sound.py`. The README's folder tree doesn't list `sound.py`, so add it there if you want the docs to match.

Summary of delivered changes

- **Task 1, direction and collision fairness:** `game/snake.py` (updated).
- **Task 2, Game Over screen with final score:** `game/game_engine.py` (updated).
- **Task 3, Easy/Medium/Hard restart and Esc/Q exit:** `game/game_engine.py` (updated).

- **Task 4, generated sound effects with no-audio fallback:** `game/sound.py` (new) and `game/game_engine.py` (updated).

Unchanged throughout: `main.py`, `game/food.py`, `requirements.txt`.